

Prefect: Why Only Orchestrate LLM Workflows?



Large Language Models (LLMs) have become an integral part of AI-driven applications, powering everything from chatbots to autonomous agents. However, working with LLMs is not just about inference; there are multiple processes involved, such as data ingestion, preprocessing, fine-tuning, evaluation, and deployment. As these workflows grow in complexity, manually managing them becomes impractical. This is where workflow orchestration tools like Prefect come into play.

Why Do We Need Workflow Orchestration for LLMs?

The lifecycle of an LLM is not a one-time process; it requires continuous refinement and monitoring. Here are some real-world scenarios where an orchestration framework like Prefect is invaluable:

1. Scheduled Fine-tuning & Retraining

LLMs need periodic fine-tuning on new data to stay relevant. Without automation, managing this can be cumbersome. Prefect enables the scheduling of fine-tuning jobs at regular intervals, automating the process end-to-end. This ensures the model is always up to date with the latest domain-specific data.

For example, an e-commerce platform may need to fine-tune its recommendation model every week based on new user interactions. Prefect can schedule these updates without manual intervention, ensuring smooth and uninterrupted improvements in model accuracy.

2. Data Pipeline Management

Preprocessing large-scale datasets for fine-tuning involves multiple steps, including:

- **Data Extraction:** Pulling data from various sources such as APIs, databases, or cloud storage.

- **Data Cleaning:** Removing inconsistencies, duplicates, and missing values.
- **Data Augmentation:** Enhancing datasets with synthetic or translated data.
- **Tokenization & Preprocessing:** Converting text into a format suitable for model training.

Prefect allows defining and automating these steps while maintaining dependencies and ensuring fault tolerance. If any step fails, it can retry the operation or alert the team without disrupting the entire pipeline.

3. Model Evaluation & Drift Detection

LLMs can degrade over time due to data drift—where the input data distribution shifts from the original training distribution. To combat this, periodic evaluation is crucial. Prefect can:

- Automate evaluation runs on a scheduled basis.
- Compute accuracy, perplexity, or other relevant metrics.
- Compare new model performance against a baseline.
- Trigger alerts or rollbacks if the model degrades beyond an acceptable threshold.

This is particularly useful in applications like fraud detection, where evolving fraud patterns require continuous retraining and evaluation.

4. Inference Pipeline Optimization

Deploying LLMs at scale involves optimizing inference pipelines to balance speed, cost, and performance. Prefect can orchestrate:

- **Model Loading & Caching:** Automatically reloading models based on demand.
- **A/B Testing:** Routing different requests to different model versions.
- **Latency Monitoring:** Logging response times and optimizing model serving strategies.

For example, an AI customer support system might use Prefect to dynamically switch between smaller, faster models for routine queries and larger, more complex models for intricate questions.

5. Multi-step Agent Workflows

When building AI agents that rely on multiple model calls—such as retrieval-augmented generation (RAG) systems—Prefect ensures all steps execute sequentially or in parallel as needed, handling retries and failures gracefully. This is essential when combining multiple APIs, embeddings, and language models in a single workflow.

Prefect's Edge Over Traditional Orchestration Tools

While tools like Apache Airflow are widely used for workflow automation, Prefect offers several advantages, particularly for AI and LLM-based workflows:

- **Dynamic DAGs:** Unlike Airflow's static DAGs, Prefect allows dynamic workflow definitions, making it more flexible for AI experimentation.
- **Built-in Resilience:** Prefect's state management ensures that if a task fails, it can retry or restart from the last successful checkpoint.
- **Hybrid Execution Model:** Prefect can run both on-premise and in the cloud, giving data teams more flexibility.
- **Developer-friendly API:** Prefect's Pythonic syntax makes it easy to integrate into existing ML pipelines.

Example: Automating LLM Fine-tuning with Prefect

Here's a more detailed example of how Prefect can automate periodic fine-tuning of an LLM:

```

from prefect import flow, task
import os

@task(retries=3, retry_delay_seconds=60)
def load_new_data():
    # Fetch new training data
    print("Fetching latest dataset...")
    return "path/to/new/data.csv"

@task(retries=3, retry_delay_seconds=120)
def fine_tune_model(data_path):
    # Fine-tune the LLM on the latest dataset
    print(f"Fine-tuning model with data from {data_path}")
    # Fine-tuning code
    return "path/to/fine-tuned-model"

@task
def evaluate_model(model_path):
    # Run evaluation metrics after fine-tuning
    print(f"Evaluating model performance for {model_path}")
    # Evaluation code

@flow
def llm_finetuning_workflow():
    data_path = load_new_data()
    model_path = fine_tune_model(data_path)
    evaluate_model(model_path)

if __name__ == "__main__":
    llm_finetuning_workflow.serve(name="llm-retraining",
        cron="0 0 * * 6", # runs every Saturday at midnight
    )

```

In this code,

- `@task(retries=n, retry_delay_seconds=k)`
 - Marks the function as a Prefect Task.
 - Retries `n` times if the task fails.
 - Waits `k` seconds before retrying.
- `@flow`
 - Marks a function as a Prefect Flow, which is a high-level container for tasks.
- `.serve(name="llm-retraining", cron="0 0 * * 6")`
 - Runs the flow automatically every Saturday at midnight.
 - Keeps it continuously available for execution.

Running the code

To execute the Prefect workflow defined in the <filename>.py file, we need to run two key commands:

1. prefect server start

- This starts the **Prefect server**, which acts as the central hub for managing and monitoring workflows.
- It provides a **UI** and a **backend** to track workflow executions, logs, and scheduling.
- The server can be accessed at `http://127.0.0.1:4200` or `http://localhost:4200`.
- The terminal should be kept running always.

```
(genai) C:\Users\Chirag\Desktop\TMLC\Generative AI course\prefect-demo>prefect server start
```

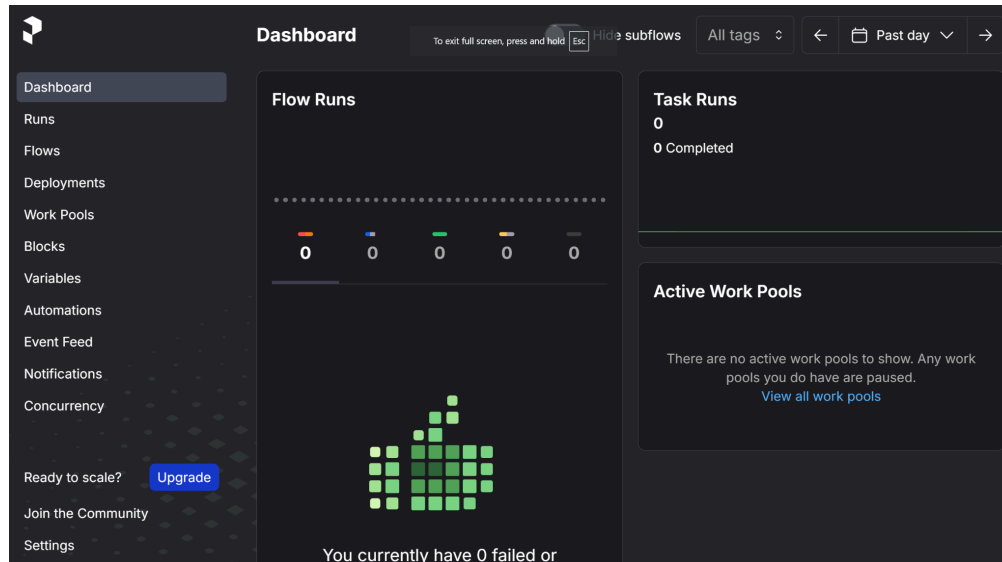
-	\	-	\	_		_	/	_		-	
_	/		/	_		_		(	_		
-				\	_		_	\			

Configure Prefect to communicate with the server with:

```
prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
```

View the API reference documentation at <http://127.0.0.1:4200/docs>

Check out the dashboard at <http://127.0.0.1:4200>



2. python <filename>.py

- Executes the orchestrator.py script, which contains the **Prefect flow and tasks**.
- The .serve() function registers the workflow with the Prefect server and ensures it runs on a schedule.
- Prefect server must be running** before executing the script.

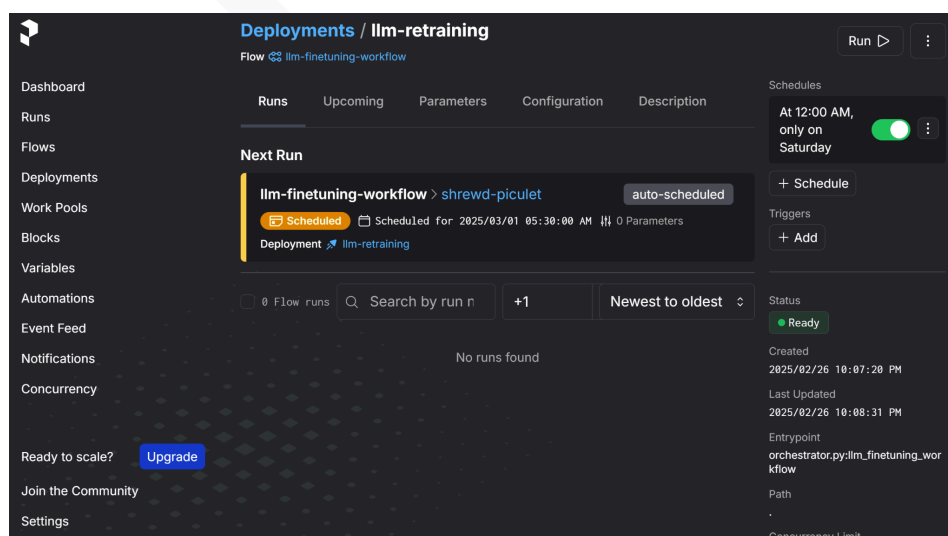
```
(genai) C:\Users\Chirag\Desktop\TMLC\Generative AI course\prefect-demo>python orchestrator.py
Your flow 'llm-finetuning-workflow' is being served and polling for scheduled runs!

To trigger a run for this flow, use the following command:

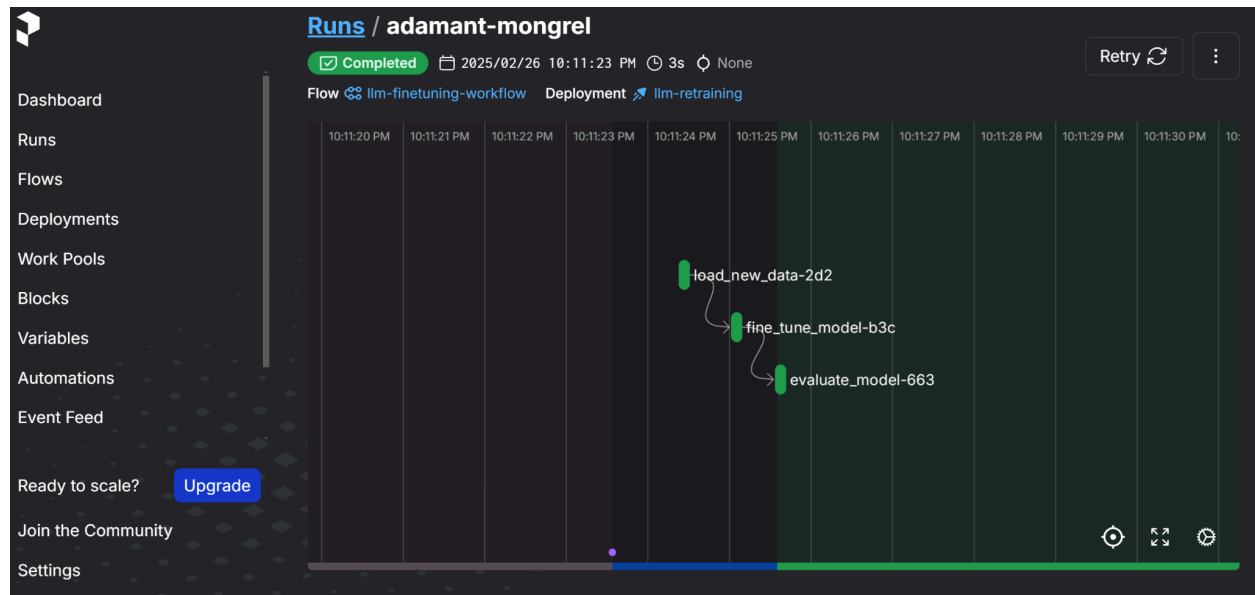
$ prefect deployment run 'llm-finetuning-workflow/llm-retraining'

You can also run your flow via the Prefect UI: http://127.0.0.1:4200/deployments/deployment/9d388530-f163-4452-a82c-0ef50fc0f7af
```

Now on the server UI go to deployments and check the workflow that has been created



To trigger a run manually before the scheduled time click the Run button -> Quick Run which will start a run at the current instant.



Any errors during the run will be highlighted in the run and scroll down to see the logs for more information.

Final Thoughts

With LLMs evolving rapidly, managing their workflows manually is no longer sustainable. Prefect provides a robust orchestration framework that ensures automation, fault tolerance, and scalability. Whether you're fine-tuning models, optimizing inference pipelines, or building multi-agent systems, Prefect can significantly streamline your AI operations.

By integrating Prefect into your LLM workflows, you can automate routine tasks, reduce operational overhead, and ensure consistent model performance. If you're working with LLMs and haven't explored workflow orchestration yet, now is the time to do so!